

Mini Banking

Mikroservisni projekat — PDS 2025/2026

Predmet: Programiranje distribuiranih sistema

Student: Lazar Ivanovic

Tema: Mini Banking (Account, Transaction, Card, User, Notification)

Stack: Spring Boot 4.1.0, Spring Cloud 2025.1.2, Docker, RabbitMQ

Informacija	Detalj
Broj servisa	5 poslovnih + 3 infrastrukturna = 8 servisa
Opcione tehnologije	Sve 4: Docker Compose, JWT, RabbitMQ, Config Server
Bonus	Load balancing (2 instance), BCrypt, @RestControllerAdvice
Verzija Jave	17

1. Uvod

Cilj projekta je razvoj mikroservisne aplikacije koja demonstrira principe modernog distribuiranog sistema u Javi sa Spring Boot-om. Aplikacija simulira bankarski sistem sa upravljanjem korisnicima, racunima, transakcijama, karticama i obavestenjima.

Sistem se sastoji od 5 poslovnih servisa i 3 infrastrukturna servisa koji medjusobno komuniciraju sinhrono (Feign) i asinhrono (RabbitMQ). Pristup je centralizovan kroz API Gateway sa JWT autentikacijom, a servisi se dinamicki otkrivaju kroz Eureka service discovery.

2. Arhitektura sistema

2.1 Pregled servisa

Servis	Port	Uloga
eureka-server	8761	Service discovery — registar servisa
config-server	8888	Centralizovana konfiguracija (native profil)
gateway	8080	API Gateway — ulazna tacka, JWT, load balancing
user-service	8081	Upravljanje korisnicima, login (generise JWT)
account-service	8082	Bankovni racuni, deposit/withdraw
transaction-service	8083	Transakcije, Resilience4j, RabbitMQ publisher
card-service	8084	Kartice (DEBITNA/KREDITNA), block/unblock
notification-service	8085	Obavestenja (RabbitMQ consumer)

2.2 Komunikacija izmedju servisa

Sinhrona komunikacija (OpenFeign):

- transaction-service -> account-service: deposit/withdraw (sa Resilience4j zastitom)
- card-service -> account-service: proverda racun postoji pri izdavanju kartice
- account-service -> user-service: proverda korisnik postoji pri kreiranju racuna
- account-service -> card-service, transaction-service: agregacioni endpoint (overview)

Asinhrona komunikacija (RabbitMQ):

- transaction-service (publisher) salje TransactionExecuted event posle uspesne transakcije
- notification-service (consumer) prima event i kreira obavestenje sa idempotentnoscu (eventId)

2.3 Agregacioni endpointi

- GET /api/accounts/{id}/overview — spaja podatke iz 3 servisa: racun + vlasnik (user) + kartice + transakcije
- GET /api/transactions/{id}/details — transakcija + podaci o racunu

3. Entiteti i struktura podataka

Svaki servis ima svoju H2 in-memory bazu (mem:userdb, mem:accountdb, itd.). Servisi ne dele baze — komunikacija ide isključivo kroz REST API (Feign) i poruke (RabbitMQ).

3.1 User (user-service)

Polje	Tip	Opis
id	Long	Primarni ključ, auto-generisan
ime	String	Ime korisnika
prezime	String	Prezime korisnika
email	String	Email adresa (@Email validacija)
telefon	String	Broj telefona
username	String	Korisnicko ime (unique)
password	String	Lozinka (BCrypt hes, ne plain tekst)

3.2 Account (account-service)

Polje	Tip	Opis
id	Long	Primarni ključ
userId	Long	Referenca na korisnika (provera kroz Feign)
brojRacuna	String	Generisan na serveru (265-XXXXXXXXXX)
stanje	BigDecimal	Pocetno 0, menja se kroz transakcije
vrstaRacuna	Enum	TEKUCI / STEDNI

3.3 Transaction (transaction-service)

Polje	Tip	Opis
id	Long	Primarni ključ
accountId	Long	Referenca na racun
tipTransakcije	Enum	UPLATA / ISPLATA
iznos	BigDecimal	Iznos transakcije (max limit iz Config Servera)
vreme	LocalDateTime	Vreme izvršenja
statusTransakcije	Enum	USPESNA / NEUSPESNA

3.4 Card (card-service)

Polje	Tip	Opis
id	Long	Primarni ključ
accountId	Long	Referenca na racun (provera kroz Feign)
brojKartice	String	Generisan na serveru (16 cifara)

Polje	Tip	Opis
tipKartice	Enum	DEBITNA / KREDITNA
statusKartice	Enum	AKTIVNA / BLOKIRANA (block/unblock endpointi)
limit	BigDecimal	Limit kartice (@Column name=limit_iznos)

3.5 Notification (notification-service)

Polje	Tip	Opis
id	Long	Primarni ključ
eventId	String	Jedinstveni ID događaja (idempotentnost)
accountId	Long	Racun na kome se desila transakcija
poruka	String	Tekst obavještenja
vreme	LocalDateTime	Vreme kreiranja obavještenja

4. Koriscene tehnologije

4.1 Eureka (Service Discovery)

Eureka server (port 8761) služi kao centralni registar servisa. Svi poslovni servisi i gateway se registruju kao Eureka klijenti pri pokretanju. Feign i Gateway koriste Eureka za dinamičko otkrivanje adresa servisa (po imenu, bez hardkodovanih URL-ova). Self-preservation je isključen za development.

4.2 Spring Cloud Gateway

Gateway (port 8080) je centralna ulazna tačka sistema. Koristi MVC varijantu (spring-cloud-starter-gateway-server-webmvc). Rute su definisane u YAML-u sa lb:// prefiksom za load-balanced rutiranje kroz Eureka. Na Gateway-u je implementiran JWT filter za autentikaciju.

4.3 OpenFeign

Feign omogućava sinhronu komunikaciju između servisa — deklarativni HTTP klijent (interfejs bez implementacije, Spring sam generise mrezne pozive). Servisi se nalaze po imenu kroz Eureka (@FeignClient(name="...")). Lokalni DTO-ovi se koriste za mapiranje odgovora (servisi ne dele Java kod).

4.4 Resilience4j (Circuit Breaker + Retry)

Primenjen na transaction-service -> account-service poziv (najkritičniji — finansijska operacija). Retry pokušava 3 puta sa pauzom od 2s. Circuit Breaker koristi sliding window od 5 poziva; ako više od 50% padne, otvara se na 10s. Poslovni ishodi (nedovoljno sredstava, račun ne postoji) se vraćaju kao enum (IshodPromene), ne kao izuzetak — tako retry ne ponavlja poslovne odluke, a circuit breaker ih ne broji kao padove. Fallback baca ServiceUnavailableException (503). Aspect order je podešen tako da CB obmotava Retry (CB spolja) — kad je osigurac otvoren, odmah se vraća fallback bez nepotrebnog ponavljanja.

4.5 Spring Boot Actuator

Uključen na svim servisima. Izloženi endpointi: /actuator/health, /actuator/info, /actuator/metrics. Koristi se za monitoring zdravlja servisa.

4.6 springdoc-openapi (Swagger)

Swagger UI je dostupan na svakom poslovnom servisu (verzija 3.0.3 za Spring Boot 4.x). Omogućava interaktivno testiranje svih REST endpointa.

5. Opcione tehnologije

5.1 Docker Compose (opcija C)

Dockerfile postoji za svaki servis (bazni image: eclipse-temurin:17-jdk-alpine). docker-compose.yml definise svih 9 kontejnera (8 servisa + RabbitMQ) sa port mapiranjima, depends_on zavisnostima i environment override-ovima. Unutar Docker mreze kontejneri komuniciraju po imenu (eureka-server, rabbitmq, config-server), a environment varijable override-uju localhost adrese iz application.properties. Ceo sistem se dize jednom komandom: docker-compose up --build.

5.2 JWT autentikacija (opcija D)

Login endpoint (POST /api/users/login) na user-service-u proverava BCrypt-ovanu lozinku i generise JWT token (jjwt 0.12.6, HS256, trajanje 1h). Gateway ima JwtAuthFilter (OncePerRequestFilter) koji proverava token na zasticenim rutama. Javne rute: login i registracija. Tajni kljuc je identican u user-service-u (generise) i gateway-u (verifikuje). SecurityConfig sa anyRequest().permitAll() iskljucuje Spring Security default zastitu.

5.3 RabbitMQ (opcija B)

Transaction-service (publisher) salje TransactionExecuted event posle uspesne transakcije. Notification-service (consumer) prima event i kreira Notification zapis. Koristi se TopicExchange sa routing key-em "transaction.executed". JSON konverzija poruka (JacksonJsonMessageConverter). Consumer implementira idempotentnost — eventId (UUID) se proverava pre kreiranja obavestenja (existsByEventId), duplikati se preskacu.

5.4 Config Server (opcija A)

Config Server (port 8888) koristi native profil sa lokalnim fajlovima (config-repo/). Zajednicke vrednosti (Eureka adresa, Actuator exposure) su u application.properties, a poslovne (transaction.max-iznos) u transaction-service.properties. Servisi se povezuju sa optional:configserver — ako Config Server nije dostupan, koriste lokalne fallback vrednosti. Vrednost max-iznos se ucitava sa @Value i primenjuje kao limit pri kreiranju transakcije.

6. Bonus funkcionalnosti

6.1 Load balancing

Transaction-service se može pokrenuti u više instanci. U IntelliJ-u: kopija Run konfiguracije sa `--server.port=8086`. U Docker-u: `--scale transaction-service=2` sa port opsegom (8091-8092:8083). Obe instance se registruju na Eureka pod istim imenom (spring.application.name=transaction-service), a Gateway (lb://) automatski balansira zahteve (round-robin).

6.2 BCrypt hesiranje lozinki

Lozinke se nikad ne čuvaju u plain tekstu. Koristi se spring-security-crypto (BCryptPasswordEncoder). Pri registraciji: `passwordEncoder.encode()`. Pri loginu: `passwordEncoder.matches()`. U bazi (H2 konzola) vidljiv je BCrypt hes (\$2a\$10\$...), ne originalna lozinka.

6.3 Globalni exception handling

Svaki servis ima `@RestControllerAdvice` klasu (`GlobalExceptionHandler`) koja hvata: `custom NotFoundException` -> 404, `validacione greske (MethodArgumentNotValidException)` -> 400, `UsernameAlreadyExistsException` -> 409, `InsufficientFundsException` -> 400, `ServiceUnavailableException` -> 503, `InvalidCredentialsException` -> 401, `LimitExceededException` -> 400. Odgovori imaju timestamp, status, error i message polja.

7. Pokretanje sistema

7.1 Lokalno (IntelliJ)

Preduslovi: JDK 17+, IntelliJ IDEA, Docker (za RabbitMQ).

- Pokrenuti RabbitMQ: `docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
- Pokrenuti servise redom: eureka (8761) -> config (8888) -> user (8081) -> account (8082) -> transaction (8083) -> card (8084) -> notification (8085) -> gateway (8080)
- Sacekati ~10-15s da se svi registruju na Eureka

7.2 Docker Compose

Preduslovi: Docker, Docker Compose.

- Maven package za svaki servis (Lifecycle -> package)
- `docker-compose up --build` (ili sa `--scale transaction-service=2` za LB)
- Sacekati ~30-60s, testirati na localhost:8080 (Gateway)

8. Struktura projekta

8.1 Struktura foldera

```
Projekat/  
|-- eureka-server/           Service discovery  
|-- config-server/          Centralizovana konfiguracija  
|-- config-repo/            Konfiguracioni fajlovi  
|   |-- application.properties  
|   |-- transaction-service.properties  
|-- gateway/                API Gateway + JWT  
|-- user-service/           Korisnici + login  
|-- account-service/        Bankovni racuni  
|-- transaction-service/     Transakcije + Resilience4j  
|-- card-service/           Kartice  
|-- notification-service/    Obavestenja (RabbitMQ)  
|-- docker-compose.yml  
|-- README.md  
|-- Izvestaj-Mini-Banking.pdf
```

8.2 Struktura svakog poslovnog servisa

```
com.example.[ime]service/  
|-- entity/                 JPA entitet (bez validacionih anotacija)  
|-- repository/             JpaRepository interfejs  
|-- dto/                    RequestDto (bez id, sa validacijom) i ResponseDto  
|-- service/                Biznis logika, DTO<->Entity konverzija  
|-- controller/             REST endpointi, ResponseEntity, @Valid  
|-- exception/              Custom exception + GlobalExceptionHandler  
|-- client/                 Feign interfejsi (gde je potrebno)  
|-- config/                 RabbitMQ, Security (gde je potrebno)  
|-- security/               JWT (user-service i gateway)
```

8.3 Verzije

Tehnologija	Verzija
Spring Boot	4.1.0
Spring Cloud	2025.1.2
Java	17
springdoc-openapi	3.0.3
jjwt	0.12.6
Resilience4j	2.3.0
RabbitMQ	3-management (Docker)
H2 Database	In-memory (po servisu)

9. Zakljucak

Projekat uspesno demonstrira principe mikroservisne arhitekture: razdvajanje na nezavisne servise, dinamicko otkrivanje (Eureka), centralizovan pristup (Gateway), sinhronu (Feign) i asinhronu (RabbitMQ) komunikaciju, otpornost na greske (Resilience4j), bezbednost (JWT), centralizovanu konfiguraciju (Config Server) i kontejnerizaciju (Docker Compose).

Poseban akcenat je stavljen na razumevanje koncepata: zasto se poslovni izuzetci vracaju kao enum umesto da se koristi ignore-exceptions za Resilience4j, zasto se koristi odvojen bean za zasticeni poziv (proxy/AOP ogranicenje), zasto retry mora da bude na najuzem mogucnom delu koda, i zasto lozinke moraju da budu hesirane (BCrypt) a ne cuvane u plain tekstu.

Implementirane su sve 4 opcione tehnologije i vise bonus funkcionalnosti, sto pokazuje celovito razumevanje Spring Cloud ekosistema i principa distribuiranih sistema.